IPCV PROJECT · SECTION D · TECHNICAL GUIDE

Face Studio Camera

Everything the project uses, why we chose it, what it is, and exactly how each feature is built and works — with worked numeric examples.

Member	Roll no.	Main area
Anmol	20231129	Enhancement filters, the 7-stage pipeline, presets, quality metrics
Vansh Gupta	20233303	Face detection (multi-scale BlazeFace), face mesh, head pose, blinks, Haar benchmark
Shreyash Kumar Dube	20233567	Segmentation and backgrounds, AR overlays, beauty and privacy filters
Sparsh Singh	20233277	IPCV Concepts Lab, live camera, batch mode, user interface, testing, documentation

How to read this guide: Chapters 1–3 give the big picture and the tools. Chapters 4–9 explain every feature: what it does, the theory, the code that implements it, and a worked example with real numbers. Chapter 10 covers how we measure results, and Chapters 11–13 cover live video, the user interface and engineering. File and function names in `this font` are the real names in the project, so you can open the code and follow along.

Contents

1 What the project does and how a photo flows through it

2 The technologies we used, and why

3 Image basics: pixels, channels and colour spaces

4 Stage 1 — Enhancement (point operations and sharpening)

5 Stage 2 — Beauty filters (skin mask, bilateral filter, lip and eye tint)

6 Stage 3 — Background effects (segmentation and alpha compositing)

7 Stages 4 and 5 — Artistic styles, vignette and grain

8 Stages 6 and 7 — AR overlays and privacy

9 Face analysis: detection, tiling, mesh, head pose, blinks, geometry, Haar comparison

10 Measuring results: metrics, robustness test, benchmark

11 The IPCV Concepts Lab

12 Live video and batch mode

13 The pipeline engine, user interface and engineering

14 Limitations and honest notes

A Formula sheet

B Glossary

1. What the project does and how a photo flows through it

Face Studio Camera is a web app (it runs in your browser, served by Python on your own laptop) that edits photos and live webcam video. It combines two kinds of technique:

- **Classical image processing** with OpenCV and NumPy: changing brightness and contrast, blurring, sharpening, edge detection, colour conversion, compositing. These are mathematical operations on pixel values, exactly as taught in the IPCV syllabus.
- **Deep-learning computer vision** with Google MediaPipe: three small pre-trained neural networks that find faces, place 478 points on each face, and separate a person from the background. These give the app its "understanding" of the photo.

The app has six tabs:

Tab	What it is for
Studio	Edit a photo with a 7-stage stackable pipeline, presets, a before/after slider, metrics and export.
Face Analysis	Face detection, 478-point mesh, head pose, eyes and mouth state, face geometry, Haar vs MediaPipe comparison.
IPCV Lab	Eight classical syllabus topics shown live on the same photo, each with a one-line explanation.
Live Camera	15 effects on the live webcam in the browser.
Batch	Apply the current pipeline to many photos and download a ZIP.
About	Architecture, team, model-file status and library versions.

1.1 Architecture

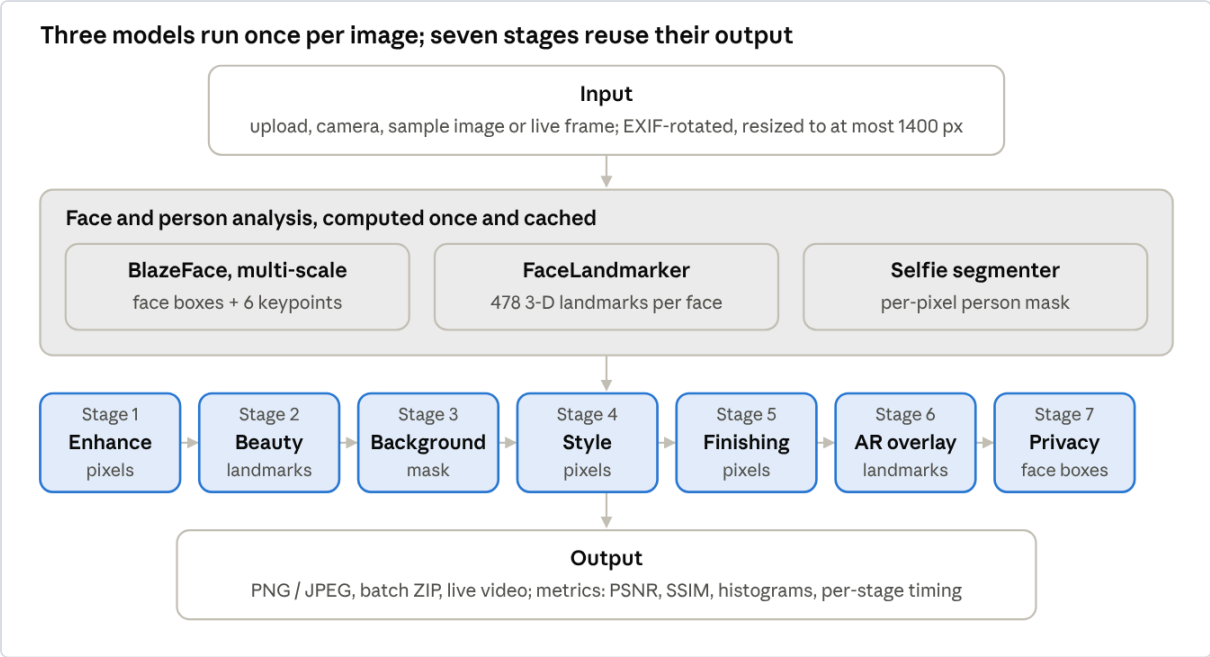


Figure 1.1 — Three models run once per image; seven stages then reuse their output.

The key design idea is **compute the expensive things once**. Running a neural network is the slowest part of the app, so the three models run once per image, and their outputs (face boxes, landmarks, person mask) are cached. When you move a slider, Streamlit re-runs the script, but the cached outputs are reused, so only the cheap pixel operations repeat. This is safe because no stage moves pixels geometrically (nothing rotates or warps the image), so the face is still in exactly the same place after every stage.

1.2 The life of one photo — step by step

1. **Input.** You upload a file, take a camera snapshot, or pick a sample. Streamlit hands the app the raw file bytes.
2. **Decode** (`decode()` in `app.py`). Pillow opens the bytes and applies `ImageOps.exif_transpose`. Phones store photos sideways and record the rotation in EXIF metadata; without this step a portrait photo would appear rotated by 90°. The image is converted to RGB, then to a NumPy array, then from RGB to BGR because OpenCV expects BGR channel order.
3. **Resize** (`filters.resize_max_dim`). If the longest side is over 1400 px it is shrunk, keeping the aspect ratio, using `INTER_AREA` interpolation (best for shrinking). Example: a 4000×3000 phone photo becomes 1400×1050, which is 8.2× fewer pixels, so every later step is about 8× faster.
4. **Analysis (only if needed).** The pipeline asks for landmarks, the mask or face boxes only when an active stage needs them. Each is computed once and cached by Streamlit, keyed on the image content.
5. **Pipeline.** `pipeline.run_pipeline` runs the enabled stages in order 1→7 and times each one.
6. **Display.** The result is shown in a before/after slider, side by side, or alone.
7. **Measure.** The Results & metrics panel compares the result with the original (PSNR, SSIM, statistics, histograms, per-stage timing).
8. **Export.** Pillow encodes the result to PNG or JPEG for download, or OpenCV writes it into `saved_images/`.

1.3 The files and what each one does

File	Lines	Responsibility
`app.py`	678	The Streamlit user interface: sidebar, 6 tabs, caching, presets, slider, metrics panel, batch, live tab.
`pipeline.py`	209	Settings, presets and the ordered 7-stage engine shared by the app, batch, live and benchmark.
`filters.py`	259	Point operations, compositing, 13 artistic styles, vignette, grain.
`face_utils.py`	342	Model wrappers: multi-scale BlazeFace, Haar baseline, FaceLandmarker with two-stage refinement, person segmenter; landmark index groups.
`face_effects.py`	444	Face frame, AR overlays, privacy, head pose, EAR/MAR, geometry, skin mask, beauty, lip and eye tint.
`ipcv_lab.py`	173	The 8 classical operations of the IPCV Lab tab.
`metrics.py`	76	PSNR, SSIM, entropy, colourfulness, sharpness, histograms.
`evaluation.py`	118	The Haar vs MediaPipe robustness experiment (12 degradations).
`live_effects.py`	125	Per-frame live processing, 15 modes, blink counter, FPS overlay.
`live_camera.py`	108	Stand-alone desktop webcam window with keyboard controls.
`benchmark.py`	148	Reproduces the report's tables as CSV files.
`check_setup.py`	86	Pre-demo self-test: packages, models, every feature, webcam.
`download_models.py`	115	Downloads the three MediaPipe models once, for offline use.
`tests/test_core.py`	89	8 automated unit tests (pytest).

2. The technologies we used, and why

Versions below are the ones a clean `pip install -r requirements.txt` installed when we tested (October 2026). Every package is free and open source and runs entirely on the laptop CPU, with no cloud service or GPU needed.

Technology	Version tested	What it is	What we use it for
Python	3.11–3.13	A general-purpose programming language with the largest ecosystem of image and AI libraries.	All of the code.
Streamlit	1.66	A library that turns a Python script into an interactive web app, with no HTML or JavaScript needed.	The whole user interface: sliders, tabs, uploads, camera snapshot, charts, downloads.
OpenCV (`cv2`)	5.0 (contrib)	The standard computer-vision library: thousands of optimised C++ functions with Python bindings.	Colour conversion, filtering, edges, thresholds, morphology, drawing, resizing, Haar cascade, the webcam window.
NumPy	2.5	Fast arrays and vectorised maths for Python.	Every image is a NumPy array; blending, masks, FFT, geometry and statistics.
MediaPipe (Tasks API)	1.1	Google's on-device machine-learning framework, with ready-trained vision models.	Face detection (BlazeFace), the 478-point face mesh (FaceLandmarker), person segmentation (Selfie Segmenter).
Pillow (PIL)	13.0	The standard Python image file library.	Decoding uploads (with EXIF rotation) and encoding PNG/JPEG downloads.
pandas	3.1	Tables (DataFrames) for Python.	Metric tables, robustness results, CSV export, histogram charts.
streamlit-webrtc	0.79	A Streamlit component that streams live video between the browser and Python over WebRTC.	The Live Camera tab.
pytest	8+	The standard Python testing tool.	8 automated tests in `tests/`.

2.1 Streamlit — the user interface

What it is. You write ordinary Python, such as `st.slider("Brightness", -100, 100)`, and Streamlit draws a slider in the browser and returns its current value to your code. A small web server (started by `streamlit run app.py`) sends the page to the browser at `http://localhost:8501`.

How it works — the rerun model. Every time you touch a widget, Streamlit re-runs the whole `app.py` from top to bottom, with each widget now returning its new value. This keeps the code simple (no event handlers), but it means anything slow must be cached, or it would repeat on every slider move. We use three Streamlit features to handle this:

- `@st.cache_resource` keeps one shared copy of heavy objects such as the loaded neural networks (`get_detector()`, `get_mesh()`, `get_segmenter()`, `get_haar()`). Loading a model takes hundreds of milliseconds, so it is done once per app run.
- `@st.cache_data` remembers function results keyed by their inputs. `_faces(img, multiscale, tile_thr)`, `_landmarks(...)` and `cached_mask(img)` hash the image array, so the same photo is analysed only once.
- `st.session_state` is a dictionary that survives reruns. Every pipeline slider is stored under a key such as `s_brightness`, which is how presets and Reset can change many sliders at once (Chapter 13).

Why Streamlit

A desktop GUI toolkit (Tkinter, PyQt) would need far more code for the same interface and could not easily show charts, tables and downloads. A full web stack (Flask + HTML + JavaScript) would take days. Streamlit gave us a professional-looking app in pure Python, and the same app can later be deployed online.

2.2 OpenCV and NumPy — the image-processing engine

What they are. In OpenCV an image is simply a NumPy array of shape `(height, width, 3)` and type `uint8` (whole numbers 0–255). A 1400×1050 photo is an array of 1050 × 1400 × 3 = 4,410,000 numbers. OpenCV functions take such arrays and return new ones; NumPy lets us do arithmetic on whole arrays at once, without writing loops over pixels.

Why vectorised maths matters. A Python `for` loop over 4.4 million values takes seconds; the same NumPy expression (`img * 0.5`) runs in compiled C code in a few milliseconds.

A trap we had to handle: uint8 overflow

In `uint8`, 250 + 10 does not give 260; it wraps around to 4. So before adding or multiplying, our filters convert to a wider type (`int16` or `float32`), do the maths, then `np.clip(…, 0, 255)` and convert back. You can see this pattern in almost every function in `filters.py`.

Why OpenCV

It is the industry standard, it is what the IPCV syllabus uses, it is fast (optimised C++), and it has every operation we needed in one library: filters, colour spaces, morphology, Canny, CLAHE, stylisation, drawing, video capture and the Haar cascade.

2.3 MediaPipe — three pre-trained neural networks

What it is. MediaPipe is Google's framework for running small machine-learning models on ordinary devices. We use its **Tasks API** (`mediapipe.tasks.python.vision`), the current, supported interface. Each model is a TensorFlow Lite file (`.tflite`) or a bundle of several (`.task`, which is a zip). They run on the CPU in a few milliseconds each.

Model (file)	What goes in	What comes out	Used by

BlazeFace short-range `blaze_face_short_range.tflite`	The image, resized internally to 128×128	For each face: a bounding box, a confidence score (0–1) and 6 keypoints (eyes, nose tip, mouth, ear tragions)	Detection tab, privacy mode, multi-scale tiling, guiding the mesh
FaceLandmarker `face_landmarker.task`	The image	For each face: 478 points with x, y and a relative depth z (468 face-surface points plus 10 iris points)	AR overlays, beauty, lips, eyes, head pose, EAR, MAR, geometry, mesh drawing
Selfie Segmenter `selfie_segmenter.tflite`	The image (resized internally to a small square)	A per-pixel confidence mask; the person channel is close to 1 on the person and close to 0 on the background	Portrait blur, background replace, colour splash

How a CNN detector like BlazeFace works, in short. A convolutional neural network slides small learned filters over the image, layer after layer, building up from edges to eyes to whole faces. BlazeFace was designed by Google for mobile phones: it uses a lightweight network and predicts, for a fixed grid of "anchor" positions, whether a face is there and how to adjust the box. Overlapping predictions are then merged. It was trained on a large set of labelled faces, so it has learned what faces look like under many poses and lighting conditions.

Why MediaPipe instead of the alternatives

- **Haar cascades (OpenCV):** about 20 years old, sensitive to head rotation, and give no landmarks or calibrated confidence. We kept Haar only as a measured baseline (Section 9.7).
- **dlib:** 68 landmarks instead of 478, often hard to install on Windows (it needs a C++ compiler), and no segmentation model.
- **Cloud APIs (Google, AWS, Azure):** need internet, an account and billing, and send photos to a server. That is useless for an offline lab demo and bad for privacy.
- **Training our own network:** impossible in the time available, and it would need thousands of labelled images.

MediaPipe gave us three accurate models, free, offline and CPU-fast, through one consistent API.

2.4 The Haar cascade — our classical baseline

What it is. The Viola–Jones detector (2001) and OpenCV's `haarcascade_frontalface_default.xml`. It works in four ideas:

1. **Haar features:** a feature is the sum of pixels in white rectangles minus the sum in black rectangles. For example, an eye region is darker than the cheek below it, so "top rectangle minus bottom rectangle" is strongly negative on an eye.
2. **Integral image:** a pre-computed table where each entry holds the sum of all pixels above and to the left. The sum of any rectangle then needs only 4 lookups, A − B − C + D, however large the rectangle is. This is what made Haar fast enough for 2001 computers.

3. **AdaBoost:** from about 160,000 possible features in a 24×24 window, training picks the few hundred that best separate faces from non-faces.
4. **Cascade:** the features are arranged in stages. Most background windows are rejected by the first cheap stages, so only face-like windows pay for the full test.

How we call it (`HaarFaceDetector.detect`): the image is converted to grey, histogram-equalised (to reduce lighting effects), and scanned with `detectMultiScale(scaleFactor=1.1, minNeighbors=5, minSize=…)`. `scaleFactor=1.1` means the window grows by 10% at each scale; `minNeighbors=5` means a face must be found by at least 5 overlapping windows, which removes many false alarms.

Why it is bundled. OpenCV 5, which MediaPipe now installs, no longer ships the cascade XML files, so we include `models/haarcascade_frontalface_default.xml` (BSD licence) in the project.

2.5 The smaller pieces

- **Pillow** decodes uploads, because it understands EXIF orientation (OpenCV's `imdecode` ignores it, so phone photos would come out sideways), and encodes downloads with a chosen JPEG quality (95).
- **pandas** builds the tables shown with `st.dataframe`, the histogram charts (`st.line_chart`) and the CSV exports.
- **streamlit-webrtc** (built on **aiortc** and **PyAV**) carries live webcam frames from the browser to Python and back (Chapter 12).
- **pytest** runs `tests/test_core.py`: every style keeps the image shape, zero settings change nothing, brightness moves the right way, PSNR/SSIM give known values, compositing respects the mask, every lab function runs, presets use valid keys, and (when models are present) the face models give sensible results on the sample.

3. Image basics: pixels, channels and colour spaces

3.1 Pixels and BGR order

A colour pixel is three numbers from 0 to 255. OpenCV stores them in the order **Blue, Green, Red (BGR)** for historical reasons, while Pillow, browsers and Streamlit use RGB. Our code works in BGR internally and converts at the edges (`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` before display). Forgetting this swap makes faces look blue, a classic bug.

Worked example

The pixel we will use throughout is BGR (200, 150, 100), which is RGB (100, 150, 200): a medium sky blue.

3.2 Why we use other colour spaces

Space	Channels	Why it is useful	Where we use it
Greyscale	One brightness value, Y = 0.299R + 0.587G + 0.114B	Most structure (edges, shapes) is in brightness; one channel is 3× less data. Green weighs most because the eye is most sensitive to it.	Edges, thresholds, Haar, SSIM, sketch, entropy
HSV	Hue (colour, 0–179 in OpenCV), Saturation (purity), Value (brightness)	Separates "which colour" from "how bright", so we can change one without the other.	Brightness (V), saturation (S), lip and eye recolouring (H and S)
LAB	L* lightness, a* green–red, b* blue–yellow	Designed so that equal number changes look like equal visual changes; L* is pure lightness.	CLAHE auto-enhance (on L* only), the Lab tab
YCrCb	Y luma, Cr and Cb colour differences	Used by JPEG and video; skin tones cluster tightly in Cr/Cb.	The Lab tab (for comparison)

Worked example — the same pixel in HSV

BGR (200, 150, 100) converts to HSV (105, 127, 200): hue 105 (blue, on OpenCV's 0–179 scale), saturation 127 (about half), value 200 (the largest channel).

3.3 Resizing

Shrinking averages many source pixels into one target pixel. `INTER_AREA` does exactly that and avoids the jagged moiré patterns other methods produce when shrinking. When we enlarge a face crop for the mesh (Section 9.3) we use `INTER_CUBIC`, which fits smooth curves through neighbouring pixels. For pixelation we deliberately use `INTER_NEAREST` to get the blocky look.

4. Stage 1 — Enhancement

These are **point operations**: each output pixel depends only on the same input pixel, not on its neighbours (sharpening and CLAHE are the exceptions, and are explained separately). They live in `filters.py` and run in this order inside the "1 Enhance" stage: auto-enhance → brightness → contrast → saturation → gamma → sharpness → warmth. A slider at its neutral value (0, or gamma 1.0) returns the image unchanged, so the stage costs nothing when unused.



Figure 4.1 — The enhancement controls on the sample portrait.

4.1 Brightness — `adjust_brightness(image, value)`

What it does: makes the photo lighter or darker without changing its colours. **How:** convert to HSV, add `value` (−100…+100) to the V channel, clip to 0–255, convert back. Working in V rather than adding to B, G and R separately keeps the hue: adding 20 to all three channels of a saturated colour would wash it out.

```
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV).astype(np.int16)
hsv[:, :, 2] = np.clip(hsv[:, :, 2] + value, 0, 255)
return cv2.cvtColor(hsv.astype(np.uint8), cv2.COLOR_HSV2BGR)
```

Worked example

Pixel BGR (200, 150, 100), HSV (105, 127, 200). With brightness +20, V becomes 220. Converting back gives BGR (220, 165, 110): every channel grows by the same 10% (220/200), so the colour stays the same blue, only lighter.

4.2 Contrast — `adjust_contrast(image, value)`

What it does: stretches values away from mid-grey (128), so darks get darker and lights get lighter. **How:** the standard contrast-correction factor:

$$F = 259(c + 255) / (255(259 - c)) \qquad out = F \cdot (in - 128) + 128$$

Worked example

c = 40 gives F = 259 × 295 / (255 × 219) = 1.368. A pixel at 100 becomes 1.368 × (100 − 128) + 128 = 89.7; a pixel at 200 becomes 226.5; a pixel at exactly 128 stays 128. Negative c gives F < 1, which flattens the image.

4.3 Saturation — `adjust_saturation(image, value)`

Multiplies the HSV S channel by (1 + value/100). A multiplication (rather than an addition) keeps greys grey: S = 0 stays 0.

Worked example

Saturation +50: S goes from 127 to 127 × 1.5 = 190, and the pixel becomes BGR (200, 125, 51): a more vivid blue at the same brightness.

4.4 Gamma — `adjust_gamma(image, gamma)`

What it does: brightens or darkens the mid-tones while keeping pure black and pure white fixed. **How:** the power law $s = 255 \times (r/255)^{1/\gamma}$. Instead of computing a power for millions of pixels, we compute it once for each of the 256 possible values and store the answers in a **lookup table (LUT)**; `cv2.LUT` then just looks each pixel up.

Input r	0	16	64	128	192	255
γ = 2.0 (brighter)	0	63	127	180	221	255
γ = 0.5 (darker)	0	1	16	64	144	255

Shadows move the most (16 → 63 with γ = 2), which is why gamma is the right tool for "lift the shadows" without blowing out the highlights.

4.5 Sharpness — `adjust_sharpness(image, amount)` (unsharp masking)

What it does: makes edges crisper. **How:** blur a copy with a Gaussian (σ = 3), then push the image away from its blurred version:

$$out = (1 + a) \cdot I - a \cdot G_\sigma(I)$$

On a flat region the image equals its blur, so nothing changes. Near an edge they differ, and that difference (the "detail") is amplified. A negative `a` moves toward the blur, which softens instead.

Worked example

Just on the bright side of an edge, I = 100 and the blurred value is 90. With a = 1: out = 2 × 100 − 90 = 110, so the bright side gets brighter (and the dark side darker), and the edge looks sharper. On a flat patch where I = 90 and the blur is 90: out = 180 − 90 = 90, unchanged.

It is implemented as one call, `cv2.addWeighted(image, 1 + a, blurred, −a, 0)`, which also saturates the result to 0–255.

4.6 Warmth — `adjust_warmth(image, value)`

Adds `value` to the red channel and subtracts it from blue (warmer, like sunset), or the opposite for negative values (cooler, like shade). Example: BGR (200, 150, 100) with warmth +30 becomes (170, 150, 130). The "Warm Tone" and "Cool Tone" styles are simply warmth +40 and −40.

4.7 Auto-enhance — `auto_enhance(image)` (CLAHE)

Background: histogram equalisation. A histogram counts how many pixels have each brightness. Equalisation maps each value through the cumulative distribution (CDF), s = 255 × CDF(r), so that the output uses the whole 0–255 range evenly.

> **Worked example (global equalisation)**
> Eight pixels with values 50, 50, 50, 52, 52, 54, 60, 200 are all crowded into the dark end. `cv2.equalizeHist` maps them to 0, 0, 0, 102, 102, 153, 204, 255: the small differences between 50, 52, 54 and 60 become large, visible steps.

Why CLAHE instead. Global equalisation can look harsh and amplifies noise in flat areas. **CLAHE** (Contrast-Limited Adaptive Histogram Equalisation) splits the image into an 8×8 grid of tiles, equalises each tile separately (adaptive), and caps each histogram bar at a clip limit (2.0), spreading the excess evenly (contrast-limited). Tiles are blended with bilinear interpolation so no seams show.

Why on the L channel of LAB. If we equalised B, G and R separately, the colours would shift. We convert to LAB, apply CLAHE to L (lightness) only, convert back, then add +12 saturation to compensate for the slightly flatter look.

```
l, a, b = cv2.split(cv2.cvtColor(image, cv2.COLOR_BGR2LAB))
l = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(l)
out = cv2.cvtColor(cv2.merge([l, a, b]), cv2.COLOR_LAB2BGR)
return adjust_saturation(out, 12)
```

5. Stage 2 — Beauty filters

All beauty effects need to know **where the skin, lips and eyes are**. That comes from the 478 face landmarks (Chapter 9). The principle is: build a mask, process a copy of the whole image, then blend the processed copy in only where the mask says so:

$$\text{out} = (1 - \text{m}) \cdot \text{image} + \text{m} \cdot \text{processed} \quad (\text{m between 0 and 1 per pixel})$$

This is `_blend()` in `face_effects.py`, used by every face effect.



Figure 5.1 — Original, the skin mask (white = smoothed), and the result. The eyes, brows and lips stay sharp because they are holes in the mask.

5.1 Building the skin mask — `skin_mask(shape, lm)`

1. **Face area:** fill the polygon through the 36 face-oval landmarks (`FACE_OVAL`) with 1s.
2. **Holes:** fill the polygons of both eyes (16 points each), both eyebrows (10 each) and the outer lips (20), and combine them with `np.maximum`.
3. **Grow the holes** with morphological **dilation** (a k×k square, k = 0.12 × the inter-ocular distance d, made odd), so a safety margin of skin around the eyes and lips is also left untouched. For our sample, d = 43.4 px gives a 5×5 kernel.
4. **Subtract** holes from the face area and clip to 0–1.
5. **Feather** with a Gaussian blur (σ = half of 0.15 d, which is 3 px here) so the mask fades out softly and no edge shows.

Every size is a fraction of `d`, so the mask fits a face that is 40 px wide or 400 px wide equally well. This scale-invariance trick is used in all face effects.

5.2 Skin smoothing — `beautify()` with the bilateral filter

The problem with an ordinary blur: a Gaussian blur smooths pores, but also smears the jaw line and nostrils. **The bilateral filter** averages each pixel with its neighbours, but weights each neighbour by two things: how close it is (spatial weight) and how similar its colour is (range weight):

$$\text{w} = \exp(-\text{distance}^2 / 2\sigma_s^2) \cdot \exp(-\Delta\text{intensity}^2 / 2\sigma_r^2)$$

Neighbours across a strong edge differ a lot in intensity, get a tiny weight, and so the edge survives.

Worked example (range weights)

Our setting `smooth = 0.6` gives $\sigma_r = 40 + 50 \times 0.6 = 70$ and diameter $d = 9 + 6 \times 0.6 = 12$ px. A neighbour that differs by 5 levels (pore texture) gets weight 0.997, so it is fully averaged and the texture disappears. One differing by 60 gets 0.693, and one differing by 150 (a real edge such as the jaw against the background) gets only 0.101, so it barely mixes in. With a smaller σ_r of 20, the weights would be 0.969, 0.011 and 0.000: even stronger edge preservation, but less smoothing.

The smoothed copy is optionally brightened (`convertScaleAbs(beta=brighten)`), then blended in with m = skin mask × min(1, smooth + 0.3 when brightening).

5.3 Lip tint — `tint_lips()`

Mask: the outer-lip polygon minus the inner-lip polygon, so an open mouth's teeth are not coloured, feathered by σ = max(1, 0.02 d). **Recolouring** (`_recolor`): convert to HSV, replace the hue with the target colour's hue, raise the saturation to at least 80% of the target's, and keep V (brightness) exactly. Because V carries the light and shadow, the lips keep their natural texture and shine; only the colour changes.

Worked example

The default lip colour #c8324b is HSV (175, 191, 200). A lip pixel BGR (60, 70, 150), HSV (3, 153, 150), becomes HSV (175, 153, 150): new hue, saturation kept (153 is already at least 0.8 × 191 = 153), same brightness. That converts to BGR (75, 60, 150), a pinker red. With the strength slider at 0.5, the final pixel is halfway between the two.

5.4 Eye colour — `tint_eyes()`

The official landmark model gives 5 points per iris (468–472 and 473–477): the centre and 4 points on the rim. The radius is the mean distance from the centre to the rim points. A sanity check follows: the centre must lie inside the eye polygon, and the radius must be between 10% and 45% of the eye's width. If not (for example, a model without iris points), the code falls back to the eye's centroid and a radius of 22% of the eye width. The mask is the iris circle **intersected** with the eye polygon, so the eyelids are never tinted. Recolouring works exactly as for the lips.

6. Stage 3 — Background effects



Figure 6.1 — Original, the raw person mask from the selfie segmenter, the feathered mask (soft edges), and portrait-mode blur.

6.1 Person segmentation — `PersonSegmenter.get_mask()`

Segmentation means labelling every pixel: here, person or background. MediaPipe's selfie segmenter is a small CNN that outputs a **confidence mask**: a value from 0 to 1 per pixel, the probability that the pixel belongs to a person. The code converts the MediaPipe output to a NumPy `float32` array and resizes it to the photo's size if needed.

> **A bug we found and fixed**
>
> Google's documentation states that the selfie segmenter outputs two masks: background at index 0 and person at index 1. The original project took `confidence_masks[0]`, the *background* mask, so portrait mode would have blurred the person and kept the background sharp. Our code takes `masks[-1]`, the last mask, which is the person mask in the two-mask model and still the right one for older single-mask models.

6.2 Feathering — `_prepare_mask()`

A raw mask has a hard, slightly jagged boundary, which looks like a paper cut-out. We blur the mask with a Gaussian of size 2 × feather + 1 (feather 10 gives a 21×21 kernel). The mask then ramps smoothly from 1 to 0 over a few pixels, which hides small segmentation errors and mimics the gradual focus fall-off of a real lens.

6.3 Alpha compositing — the core formula

$$O(x, y) = \alpha(x, y) \cdot I(x, y) + (1 - \alpha(x, y)) \cdot B(x, y)$$

I is the photo, B is the new background, and α is the feathered mask. Implemented in `alpha_composite()` with `float32` maths and a final clip to 0–255.

> **Worked example**
>
> Inside the person α = 1, so O = I: untouched. Far in the background α = 0, so O = B. On the feathered edge, say α = 0.7 with I = 200 and B = 50: O = 0.7 × 200 + 0.3 × 50 = 155, a smooth mix.

6.4 The four background modes

Mode	B (the new background) is…	Function
Blur (portrait mode)	A Gaussian-blurred copy of the photo, kernel 2 × strength + 1 (strength 18 gives 37×37). This imitates a camera's shallow depth of field ("bokeh").	`blur_background`
Solid colour	A flat colour from the colour picker. Hex #1e2a3a becomes BGR (58, 42, 30) via `hex_to_bgr`.	`replace_background_color`
Custom image	An uploaded photo, scaled to *cover* the frame and centre-cropped (see below).	`replace_background_image`
Colour splash	A greyscale copy, so only the person stays in colour.	`greyscale_background`

Worked example — fitting a background photo

Our photo is 1400×1050 and the uploaded background is 1920×1080. The scale is max(1400/1920, 1050/1080) = max(0.729, 0.972) = 0.972, so the background becomes 1867×1050: tall enough, slightly too wide. The extra 467 px of width are cropped equally from both sides (233 px each). Taking the *max* of the two ratios guarantees there are no empty borders; the *min* would leave bars.

7. Stages 4 and 5 — Artistic styles, vignette and grain



Figure 7.1 — Eight of the 13 styles: sepia, noir, cartoon, pencil sketch, watercolour, oil paint, emboss, thermal.

All styles live in the `STYLES` dictionary in `filters.py`. Each one is a function that takes a BGR image and returns a BGR image of the same size, which is what lets the pipeline stack them freely (a unit test checks this for every style).

Style	How it is made	IPCV concept
Grayscale	`COLOR_BGR2GRAY` (Y = 0.299R + 0.587G + 0.114B), copied back to 3 channels.	Colour-space conversion
Sepia	A 3×3 colour matrix: R′ = 0.393R + 0.769G + 0.189B, G′ = 0.349R + 0.686G + 0.168B, B′ = 0.272R + 0.534G + 0.131B, via `cv2.transform` (columns reversed for BGR input). Example: RGB (100, 150, 200) becomes (192, 171, 134), a warm brown.	Linear colour transform
Noir	Grey → CLAHE (clip 3.0) → contrast +40: deep blacks and strong local contrast.	Histogram processing
Cartoon	(1) Edges: grey → 5×5 median blur → adaptive mean threshold (block 9, C 9) gives black outlines. (2) Colour: two bilateral filters flatten areas while keeping edges; colour quantisation `(c // 32) * 32 + 16` reduces each channel to 8 levels. (3) Colour AND edges.	Non-linear filtering, thresholding, quantisation
Pencil Sketch	Colour dodge: grey ÷ (255 − blurred(255 − grey)) × 256. Where the image is flat the result is white; at edges it darkens, like pencil lines.	Image arithmetic, blending
Colour Pencil	`cv2.pencilSketch` (colour output).	Edge-preserving filtering
Watercolour	`cv2.stylization` (σ_s 60, σ_r 0.45): heavy edge-preserving smoothing.	Edge-preserving filtering

Oil Paint	Recursive edge-preserving filter, then `cv2.detailEnhance` to bring back brush-like detail.	Edge-preserving filtering
Emboss	Convolution with the 3×3 kernel [[−2, −1, 0], [−1, 1, 1], [0, 1, 2]]: a directional derivative that looks like raised relief.	Spatial convolution
Edge Detect	Grey → 5×5 Gaussian → Canny (thresholds 80, 160).	Edge detection
Thermal	Grey → `applyColorMap(COLORMAP_INFERNO)`: maps brightness to a false-colour scale.	Pseudo-colouring
Cool / Warm Tone	Warmth −40 / +40 (Section 4.6).	Channel arithmetic

7.1 Vignette — `apply_vignette(image, strength)`

A vignette darkens the corners and draws the eye to the centre. We build a 2-D Gaussian as the **outer product** of two 1-D Gaussians (`getGaussianKernel` for width and height), normalise its peak to 1, and multiply the image by it. A larger strength gives a smaller σ (σ = size / (0.5 × strength)), so the fall-off is faster.

> **Worked example**
> For a 1400×1050 image, strength 1.5 keeps 87% of the brightness in the corners and 93% at the middle of the left edge. Strength 3.0 keeps only 57% in the corners. The centre is always 100%.

7.2 Film grain — `add_film_grain(image, amount)`

Adds Gaussian noise with standard deviation `amount` to every pixel (the same noise value on all three channels, like real film). The random generator is seeded (`default_rng(7)`), so the same settings always give the same picture, which matters because Streamlit re-runs the script on every slider move and changing noise would flicker.

8. Stages 6 and 7 — AR overlays and privacy

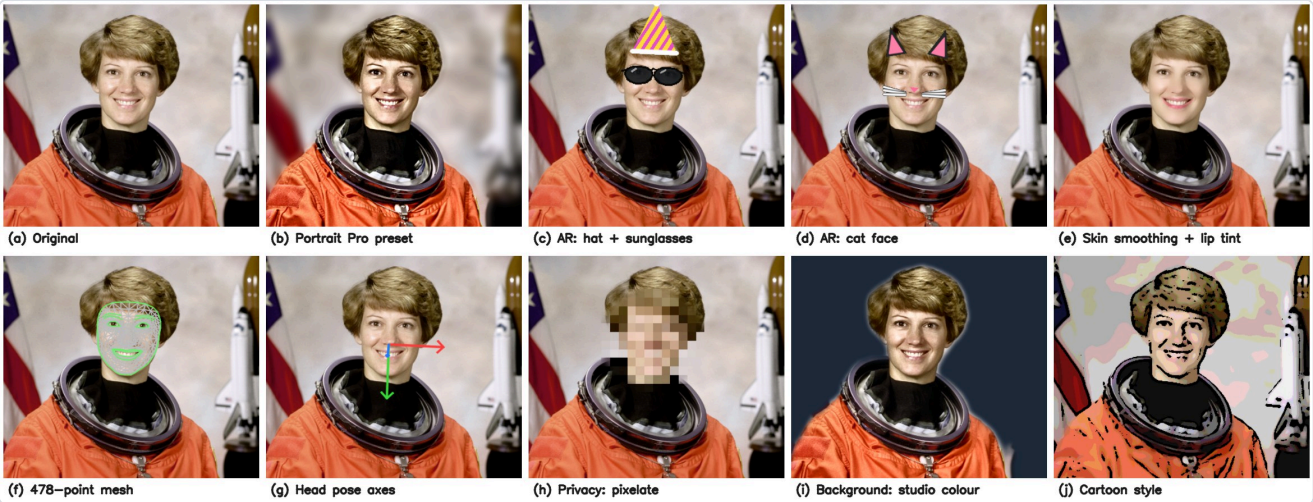


Figure 8.1 — (a) original, (b) Portrait Pro preset, (c) hat + sunglasses, (d) cat face, (e) skin smoothing + lip tint, (f) face mesh, (g) head-pose axes, (h) privacy pixelation, (i) studio backdrop, (j) cartoon.

8.1 The face-local coordinate frame — `FaceFrame`

The challenge in AR is that the same sunglasses must fit a face that is near or far (any size), anywhere in the image, and tilted. We solve this once, with a coordinate system attached to the face:

- **Origin:** the midpoint between the two eye centres (each eye centre = the mean of its 16 landmarks).
- **Unit length d:** the inter-ocular distance (centre to centre).
- **e_1 (u-axis):** unit vector from the subject's right eye to the left eye, along the eye line.
- **e_2 (v-axis):** e_1 rotated by 90°: $(-e_{1y}, e_{1x})$, pointing down toward the chin.

Every overlay is designed once, in units of d, and mapped to pixels with:

$$\text{pixel} = \text{origin} + d\cdot(u\cdot e_1 + v\cdot e_2) \qquad (\texttt{FaceFrame.to_img})$$

The inverse (`to_local`) takes a landmark and returns its (u, v): $u = (p - \text{origin})\cdot e_1/d$ and $v = (p - \text{origin})\cdot e_2/d$. That is how an overlay asks, for example, "where is the forehead on this face?"

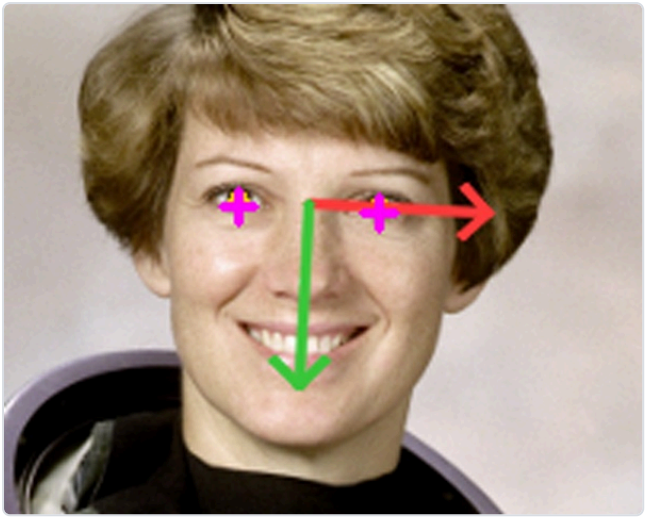


Figure 8.2 — The face frame on the sample: origin between the eyes, e_1 (red) along the eye line, e_2 (green) toward the chin, and the two sunglasses-lens centres (magenta crosses) at u = ±0.5.

Worked example (real numbers from the sample)

Origin = (225.5, 102.8), d = 43.4 px, e_1 = (0.999, 0.047), e_2 = (−0.047, 0.999), so the head is tilted by atan2(0.047, 0.999) = 2.7°.

The left lens centre is defined at (u, v) = (−0.5, 0.03):
x = 225.5 + 43.4 × (−0.5 × 0.999 + 0.03 × −0.047) = 203
y = 102.8 + 43.4 × (−0.5 × 0.047 + 0.03 × 0.999) = 103
The right lens lands at (247, 105). Note that the right lens is 2 px lower: the 2.7° tilt is followed automatically.

Landmark 10 (top of forehead) has v = −0.78: it sits 0.78 inter-ocular distances above the eye line, and the party hat uses it as its base.

8.2 How each overlay is drawn

All overlays are **procedural**: drawn from shapes with OpenCV (`fillPoly`, `polylines`, `circle`, `line`, all with anti-aliasing `LINE_AA`), with no image files. Each overlay draws onto a copy of the image (`layer`) and, at the same time, into an `alpha` map saying how opaque each drawn pixel is. Then `_blend(img, layer, alpha)` merges them. This gives semi-transparent lenses and smooth edges.

Overlay	Construction in face units
Sunglasses	Two ellipses centred at u = ±0.5, v = 0.03, radii 0.47 d × 0.33 d (about 20×14 px here), top edge flattened; filled dark at 88% opacity; frame lines of thickness 0.06 d; a bridge over the nose; arms to u = ±1.08; a thin white glare stroke on each lens at 60% opacity.
Party hat	A triangle with its base at the forehead (landmark 10's v + 0.05), 1.5 d wide, apex 1.75 d above the base; diagonal stripes clipped to the triangle by a mask (stripes AND triangle); a white pom-pom and band.
Moustache	A handlebar outline of 11 points, mirrored to make it symmetric, placed halfway between the nose base (landmark 2) and the upper lip (landmark 0). Its width follows the mouth corners (61 and 291) and its height the nose-to-lip gap, so it fits every face shape.
Cat face	A pink triangle nose at landmark 4, three whiskers per side (dark outline + white line), and two ears (dark outer triangle, pink inner) above the forehead.

Why procedural drawing instead of PNG stickers

PNG stickers need image assets (with possible copyright issues), must be resized and rotated with an affine warp, and look blurry when enlarged. Shapes drawn in the face frame are always sharp at any size, rotate for free, need no files, and are easy to explain line by line.

8.3 Privacy — `anonymize_faces(image, faces, mode)`



Figure 8.3 — Blur, Pixelate, Eye bar, Solid.

This stage uses the BlazeFace boxes (not the mesh), because it must work even on small or turned faces where a mesh may fail, and it runs **last** in the pipeline so no later stage can draw over the anonymised area.

1. **Pad** each box by 25% on every side, so hair and chin edges are covered too.
2. **Make the effect** on that region:
   - **Blur:** a Gaussian with kernel = one-third of the region size, applied twice. A single small blur can be partly reversed; this is deliberately strong.
   - **Pixelate:** shrink the region so each block is `max(6, width // 12)` pixels, then enlarge with `INTER_NEAREST`, which copies values without smoothing and makes the blocks. Example: a 103 px box padded by 25 px on each side (153 px) gives 12-px blocks, about 12 blocks across the face.
   - **Solid:** a flat dark fill.
   - **Eye bar:** a black rotated rectangle from the two eye keypoints, twice as long as the eye distance and 0.32 times as tall, rotated to the eye line's angle (`cv2.boxPoints`).
3. **Blend through an ellipse** (filled with `cv2.ellipse` and feathered with σ = width/30), so the anonymised patch is face-shaped with soft edges instead of a hard rectangle.

9. Face analysis

Note on the numbers in this chapter: they were measured on the bundled NASA sample photo during development. With the official MediaPipe models on your laptop they may differ slightly in the last digit; the method is identical.

9.1 Face detection with BlazeFace — `FaceDetector._detect_once()`

The detector returns, for each face, a dictionary `{bbox: (x, y, w, h), keypoints: [6 points], score}`. MediaPipe reports the keypoints as fractions of the image size (0–1), so the code multiplies them by width and height to get pixels, and clips the box to the image. Detections below `min_confidence = 0.5` are dropped inside MediaPipe.

> **Worked example (sample photo)**
>
> One face: box (173, 74, 103, 103), score 0.968. Keypoints: right eye (203, 102), left eye (247, 104), nose tip (223, 129), mouth (222, 149), right ear (180, 108), left ear (271, 114). The "right eye" appears on the *left* of the image because it is the subject's right.

9.2 Multi-scale tiled detection — `FaceDetector.detect()`

The problem we measured. The short-range model is built for selfies: it shrinks the whole frame to 128×128 before looking for faces. In a group photo of 864×890 px, a face 40 px tall is scaled by 128/890 = 0.144 and becomes about 6 px, far too small to recognise. On our test set, a single pass found only 2 of 20 faces.

The fix: an image pyramid of tiles. We also run the detector on overlapping crops (tiles), so that each face fills a larger share of what the model sees:

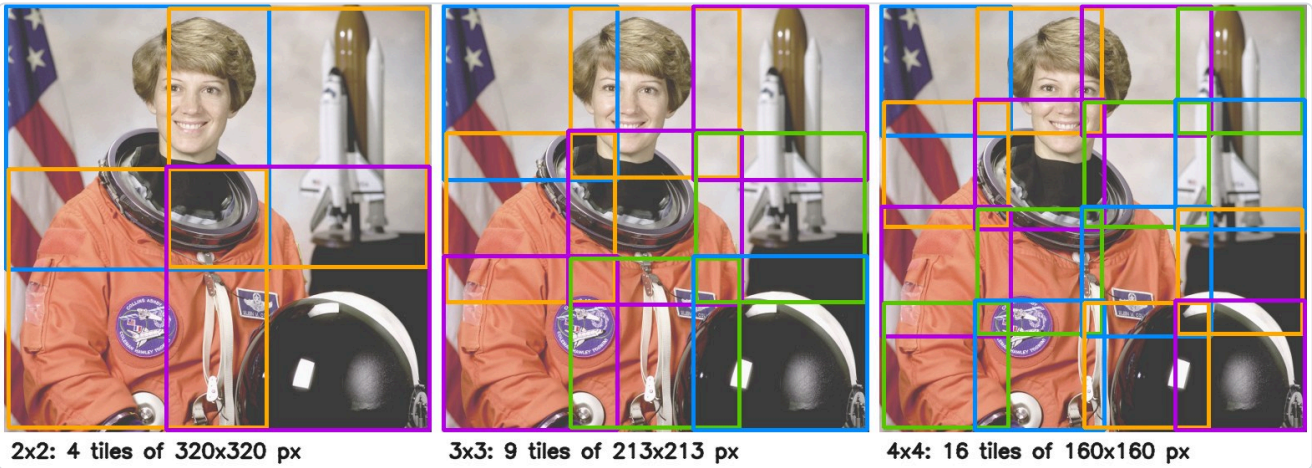


Figure 9.1 — The three tile levels on a 512×512 image. Each tile is 25% larger than a plain grid cell, so neighbouring tiles overlap and a face on a border is fully inside at least one tile.

1. Run once on the full image (catches large faces).
2. For n = 2, 3 and 4: the tile size is (W/n × 1.25) × (H/n × 1.25). Tile (i, j) is centred on grid cell (i, j) and clamped inside the image. That is 4 + 9 + 16 = 29 tiles, so 30 detector passes in total, about 135 ms per image instead of 5 ms.
3. Keep a tile's detection only if its score is at least `tile_min_score = 0.6`. Tiles see less context, so they are more easily fooled; on the original test images this threshold left no false boxes. It

can be changed with a sidebar slider.
4. Shift each tile's boxes and keypoints by the tile's top-left corner (x_0, y_0) back into full-image coordinates.
5. Merge duplicates with **non-maximum suppression** (below).

> **Worked example — what tiling buys**
> For the 864×890 group photo, the 4×4 tiles are 270×278 px. The same 40 px face is now scaled by 128/278 = 0.46 and appears about 18 px tall to the model, 3× larger than in the single pass, and big enough to detect.

Non-maximum suppression (NMS) — `nms(faces, iou_thresh=0.3)`

The same face is usually found by several tiles. NMS keeps one box per face:

1. Sort all boxes by score, highest first.
2. Take the best box and keep it. Discard every remaining box that overlaps it by IoU > 0.3, or that lies more than 60% inside it (a partial face from a tile edge).
3. Repeat with the next remaining box.

$$\text{IoU (Intersection over Union)} = \text{area}(A \cap B) \,/\, \text{area}(A \cup B)$$

> **Worked example — IoU**
> Box A = (100, 100, 50, 50) and box B = (110, 105, 50, 50). Their overlap is 40 × 45 = 1800 px². Union = 2500 + 2500 − 1800 = 3200 px². IoU = 1800 / 3200 = 0.56 > 0.3, so B is a duplicate of A and is removed (assuming A scored higher).

9.3 Two-stage face mesh — `FaceMeshDetector.landmarks(image, boxes)`

FaceLandmarker contains its own short-range detector, so it has the same small-face weakness. We therefore give it a second chance, guided by the multi-scale boxes:

1. Run FaceLandmarker on the whole image.
2. For each detected box whose centre is not inside any mesh already found: crop a square twice the box size around it, enlarge it so its longer side is at least 384 px (`INTER_CUBIC`), and run FaceLandmarker on the crop.
3. Map the points back: divide by the enlargement factor, then add the crop's top-left corner. If the crop contains several faces, keep the mesh whose centre is closest to the box centre.

Each landmark is returned as pixel coordinates (x, y) plus z, a relative depth in the same units as x. A smaller z means closer to the camera; the nose tip has the smallest z on a frontal face.

9.4 The 478 landmarks we use

Group (constant in `face_utils.py`)	Indices	Used for
Face oval `FACE_OVAL`	36 points	Skin mask outline

Eyes `LEFT_EYE`, `RIGHT_EYE`	16 points each	Face frame (eye centres), skin-mask holes, eye-tint clipping
Brows `LEFT_BROW`, `RIGHT_BROW`	10 points each	Skin-mask holes
Lips `LIPS_OUTER`, `LIPS_INNER`	20 points each	Lip-tint ring, skin-mask hole
Irises `LEFT_IRIS`, `RIGHT_IRIS`	473–477, 468–472	Eye colour (centre + 4 rim points)
EAR points `EAR_RIGHT`, `EAR_LEFT`	33,160,158,133,153,144 and 362,385,387,263,373,380	Eye aspect ratio
Single points	1 nose tip · 2 nose base · 4 nose · 10 forehead · 152 chin · 33/263 outer eye corners · 61/291 mouth corners · 0 upper lip · 13/14 inner lips · 234/454 cheeks	Head pose, AR placement, MAR, face geometry

9.5 Head pose — `head_pose(lm)`

Goal: three angles describing where the head points. Yaw = turning left or right, pitch = nodding up or down, roll = tilting toward a shoulder.

Method: build the head's own axes from 3-D landmarks (Gram–Schmidt).

1. $\mathbf{x_h}$ = the vector from eye corner 33 to eye corner 263, normalised to length 1 (across the face).
2. $\mathbf{y_h}$ = the vector from forehead 10 to chin 152, minus its component along x_h (so it is exactly perpendicular), normalised (down the face). Removing the component is the Gram–Schmidt step.
3. $\mathbf{z_h}$ = $x_h \times y_h$ (cross product): perpendicular to both, pointing into the screen.
4. R = $[x_h\ y_h\ z_h]$ is the head's rotation matrix in camera coordinates. Then:

$$\text{yaw} = \text{atan2}(-z_x, z_z) \quad \text{pitch} = \text{atan2}(z_y, \sqrt{z_x^2 + z_z^2}) \quad \text{roll} = \text{atan2}(x_y, x_x)$$

Worked example (sample photo)

x_h = (0.999, 0.047, −0.024), y_h = (−0.041, 0.976, 0.215), z_h = (0.034, −0.214, 0.976).
yaw = atan2(−0.034, 0.976) = −2.0° (almost straight ahead)
pitch = atan2(−0.214, 0.977) = −12.4° (slightly down)
roll = atan2(0.047, 0.999) = +2.7° (a slight tilt, matching the face frame in Section 8.1)

How we checked the signs: mirroring the photo flips the signs of yaw and roll (−0.7° → +1.2° and +2.5° → −2.7° in our test), and rotating the image by 20° changes roll by about 20°. The axes are drawn at the nose tip (`draw_head_pose`): x red, y green, and the "toward the camera" direction blue.

Why not cv2.solvePnP (our first version)

The textbook method fits a generic 3-D head model (6 points in millimetres) to the 2-D landmarks. We implemented it first, but it reported pitch −36° on a face looking straight at the camera, because a generic head does not match every person's proportions. The Gram–Schmidt method needs no head model; it uses MediaPipe's own depth estimates and gave −12° on the same photo. A small pitch bias remains (listed in the limitations), because a real forehead-to-chin line is not exactly vertical.

9.6 Eyes and mouth — `eye_aspect_ratio`, `BlinkCounter`, `mouth_aspect_ratio`

Eye aspect ratio (Soukupová & Čech, 2016). Six points around each eye: p1 and p4 are the corners, p2/p6 and p3/p5 are pairs above and below:

$$\mathrm{EAR} = (\|p2 - p6\| + \|p3 - p5\|) \,/\, (2\|p1 - p4\|)$$

An open eye has EAR around 0.25–0.35; when it closes the vertical distances shrink toward 0 while the width stays the same. Below 0.20 we call the eye closed. The ratio does not depend on face size, because both distances scale together.

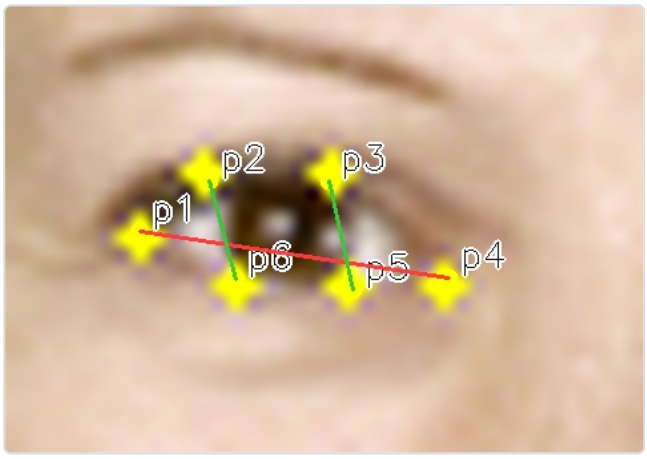


Figure 9.2 — The six EAR points on the right eye: width p1–p4 (red), heights p2–p6 and p3–p5 (green).

Worked example (sample, right eye)

p1 (194.4, 101.1), p2 (198.8, 98.0), p3 (206.3, 97.9), p4 (213.8, 104.0), p5 (207.8, 104.8), p6 (200.4, 104.2).
‖p2 − p6‖ = 6.40, ‖p3 − p5‖ = 7.06, ‖p1 − p4‖ = 19.62.
EAR = (6.40 + 7.06) / (2 × 19.62) = 0.343, so the eye is open.

Blink counter (`live_effects.BlinkCounter`) is a tiny state machine run on every video frame: while EAR < 0.20 it counts closed frames; when EAR rises again, it adds one blink *only if* the eyes were closed for at least 2 frames. This ignores single-frame glitches.

Frame	1	2	3	4	5	6	7	8	9
EAR	0.30	0.30	0.10	0.10	0.10	0.30	0.12	0.30	0.30
Closed frames	0	0	1	2	3	→0	1	→0	0
Blinks	0	0	0	0	0	**1**	1	1 (frame 7 was only 1 frame: ignored)	1

Mouth aspect ratio = distance between inner-lip points 13 and 14 ÷ distance between mouth corners 61 and 291. Above 0.35 the mouth is "open". The smiling sample scores 0.165 (closed, teeth

showing but lips nearly together).

9.7 Face geometry and symmetry — `face_geometry(lm)`

Face width = cheek 234 to cheek 454 (92 px on the sample); height = forehead 10 to chin 152 (108 px); ratio 1.17. **Symmetry:** take the midline from point 10 to point 152; for five mirrored pairs (eye corners 33/263, mouth corners 61/291, cheeks 234/454, brows 70/300, jaw 172/397) measure each point's perpendicular distance to the midline; the relative difference $|d_a - d_b| / \max(d_a, d_b)$ is 0 for a perfect mirror. Symmetry = 100 × (1 − mean difference) = 95.4% on the sample.

9.8 Haar vs MediaPipe comparison

The Face Analysis tab shows both detectors on the same photo with face counts and times, and can run the 12-condition robustness test on any uploaded photo. The method and results are in Section 10.3.

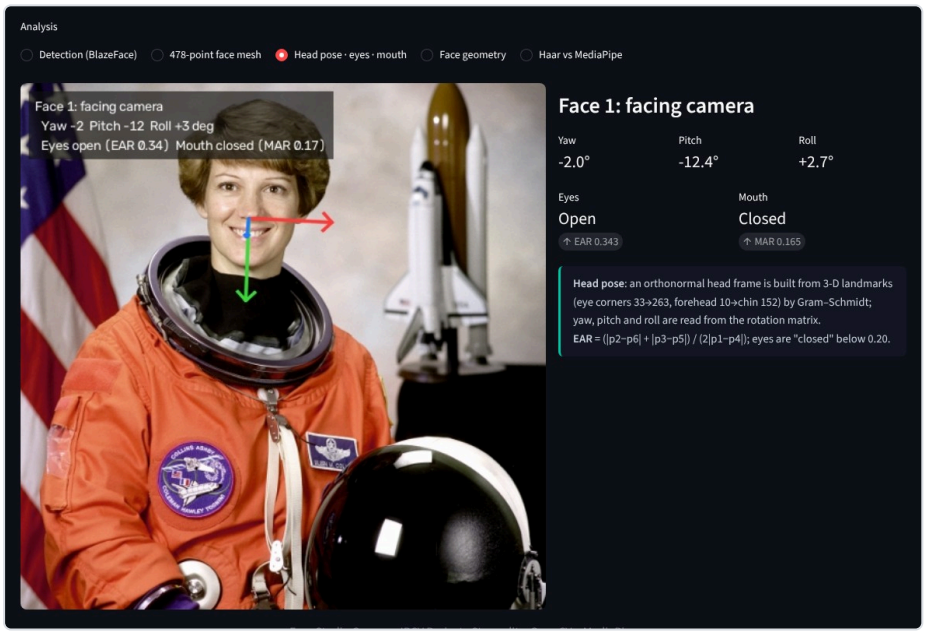


Figure 9.3 — The Head pose · eyes · mouth view in the app.

10. Measuring results

A computer-vision project needs numbers, not only pictures. `metrics.py` provides the measures shown in the Studio's *Results & metrics* panel and used by `benchmark.py`.

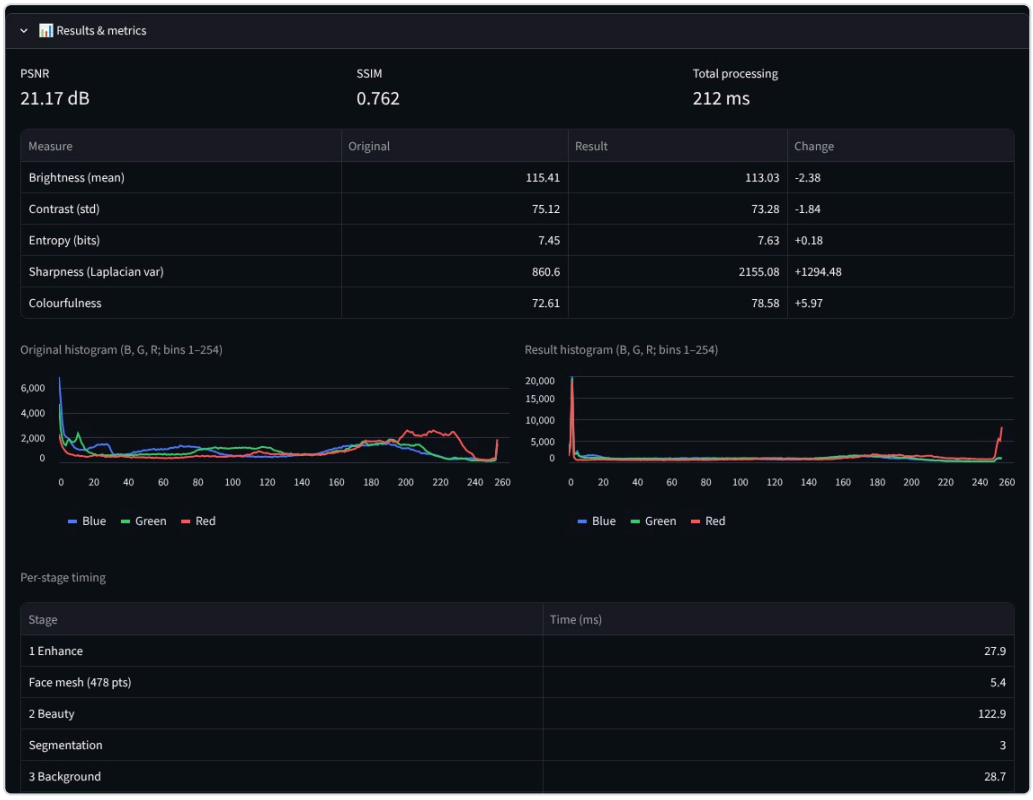


Figure 10.1 — The Results & metrics panel after the Portrait Pro preset.

10.1 Comparing two images: PSNR and SSIM

PSNR — Peak Signal-to-Noise Ratio, `psnr(a, b)`

$$\text{MSE} = \text{mean of } (a - b)^2 \text{ over all pixels} \qquad \text{PSNR} = 10 \cdot \log_{10}(255^2 / \text{MSE}) \quad \text{dB}$$

Higher means more similar; identical images give infinity (shown as "∞ (identical)"). Above about 40 dB a change is hard to see; around 20 dB it is obvious.

Worked example

A 10×10 grey patch of value 100, with the top half changed to 110: half the pixels differ by 10, so MSE = 0.5 × 10^2 = 50 and PSNR = $10 \cdot \log_{10}(65025 / 50)$ = 31.1 dB.

SSIM — Structural Similarity, `ssim(a, b)` (Wang et al., 2004)

PSNR treats every pixel error the same, but our eyes care more about structure. SSIM compares, inside an 11×11 Gaussian window (σ = 1.5) slid over the image, three things: mean brightness (μ), contrast (σ) and structure (the correlation σ_{xy}):

$$\text{SSIM} = [(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)] \,/\, [(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)]$$

with $C_1 = (0.01 \cdot 255)^2$ and $C_2 = (0.03 \cdot 255)^2$ to avoid dividing by zero. We compute it on the greyscale image and average over all windows. 1 means identical; 0 means no structural similarity. Means, variances and covariance are all computed with Gaussian blurs (for example $\sigma_{xy} = \text{blur}(a \cdot b) - \text{blur}(a) \cdot \text{blur}(b)$), which is fast.

Change applied to the sample	PSNR (dB)	SSIM	What it shows
Brightness +40	17.99	0.814	Big pixel change, but the structure is mostly intact
Gaussian blur 9×9	25.99	0.857	Small pixel change, but fine structure is lost
Gaussian noise σ = 25	20.87	0.448	Noise destroys local structure: SSIM falls sharply
Greyscale	17.88	1.000	SSIM uses luminance only, so removing colour looks "perfect" to it

The last row is an important lesson for viva questions: **PSNR and SSIM measure how much an image changed, not how good it looks.**

10.2 Describing one image: statistics and histograms

Measure	Definition	Sample value
Brightness	Mean of the greyscale image	115.4
Contrast	Standard deviation of the greyscale image	75.1
Entropy	$-\Sigma\, p_i \log_2 p_i$ over the 256-bin histogram: how much information (bits per pixel). Example: an image that is half 0 and half 255 has entropy 1 bit; four equally common values give 2 bits.	7.45 bits
Sharpness	Variance of the Laplacian (the second derivative). Sharp images have strong, varied edges, so a high variance; blur lowers it.	860.6
Colourfulness	Hasler & Süsstrunk (2003): with rg = R − G and yb = (R + G)/2 − B, $\sqrt{(\sigma_{rg}^2 + \sigma_{yb}^2)} + 0.3 \cdot \sqrt{(\mu_{rg}^2 + \mu_{yb}^2)}$	72.6

Histograms are computed per channel with `cv2.calcHist` (256 bins). The app plots bins 1–254 only: the sample has many pure-black pixels, and bin 0 would be so tall it squashed the rest of the chart flat.

10.3 The robustness experiment — `evaluation.py`

Question: under which conditions does each detector fail? **Method:**

1. Test images with known face counts: 7 classic CMU/MIT face-detection test images from OpenCV's test data (including one with no faces, to test false alarms) plus the NASA portrait, 20 faces in total. Every detected box was checked by eye.
2. Each image is transformed 12 ways (`VARIANTS`): original; rotation 15°, 30°, 45°; gamma 3 and 5 (dark) and 0.3 (over-exposed); Gaussian noise σ = 25 and 50; a 15 px blur; down-scaling to 40%; mirroring.

3. Three detectors run on each of the 84 variants: multi-scale BlazeFace, single-pass BlazeFace and Haar. For each we record faces found, whether all true faces were found, extra boxes (found − true, when positive), and time. One warm-up call runs first so the timing excludes start-up.

Result	BlazeFace single pass	BlazeFace multi-scale	Haar cascade
Faces found on the originals	2 / 20 (10%)	16 / 20 (80%)	20 / 20 (100%)
Images with all faces found at 30° rotation	2 / 7	6 / 7	1 / 7
Extra boxes over all 84 trials	0	13	7
Mean time per image	4.8 ms	138 ms	152 ms

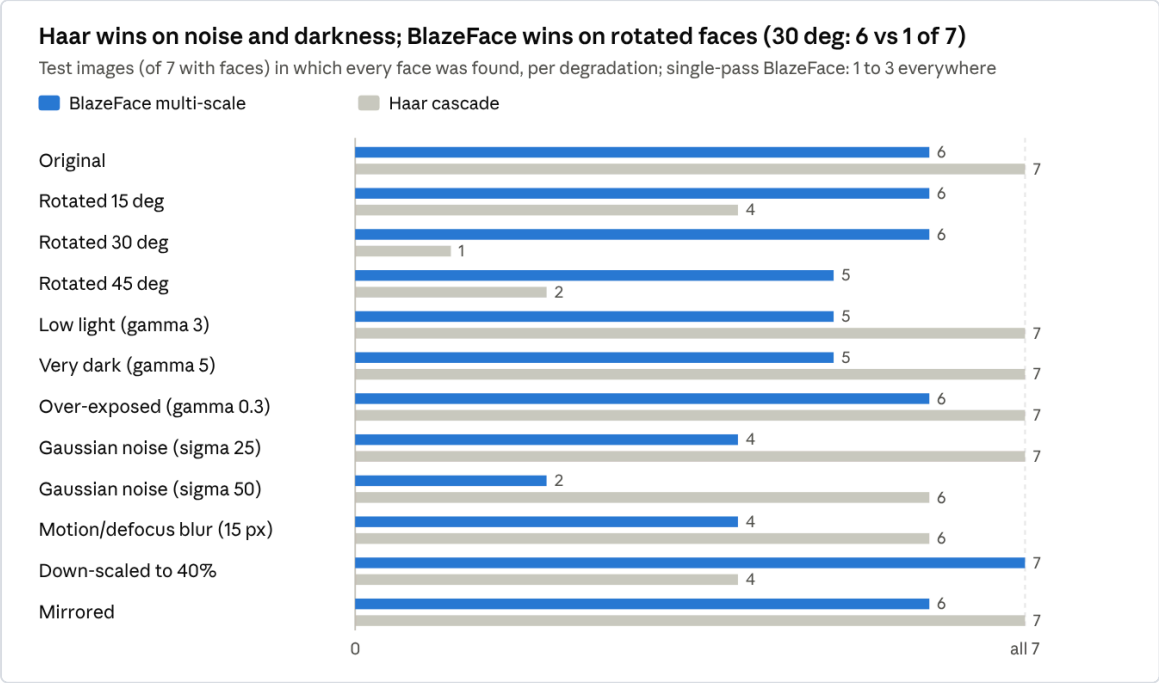


Figure 10.2 — Images (of 7 containing faces) in which every face was found, per degradation.

Interpretation. Haar's rectangle features were trained on upright frontal faces, and these classic test images are exactly that kind, so Haar wins on the originals, in darkness and under heavy noise. As soon as heads tilt, Haar breaks down, while BlazeFace keeps working. Single-pass BlazeFace is the fastest by far (about 30× faster than Haar), and tiling closes most of the small-face gap at a cost in time and a few false boxes on noisy images.

10.4 The benchmark script — `benchmark.py`

Running `python benchmark.py --images folder --gt gt.csv` regenerates every number in the report as CSV files in `docs/results/`: face counts, robustness, summary, timing (mean and standard deviation of 10 runs per operation) and quality (PSNR/SSIM per filter), plus annotated detection images for visual checking. The `gt.csv` file simply lists `file,faces`. Because timings depend on the CPU, re-running it on the demo laptop gives that laptop's own numbers.

11. The IPCV Concepts Lab — `ipcv_lab.py`

This tab shows the classical syllabus operations live on the current photo. Each function returns `(output_image, explanation)`; the explanation is printed under the picture, often with the actual parameter values used.

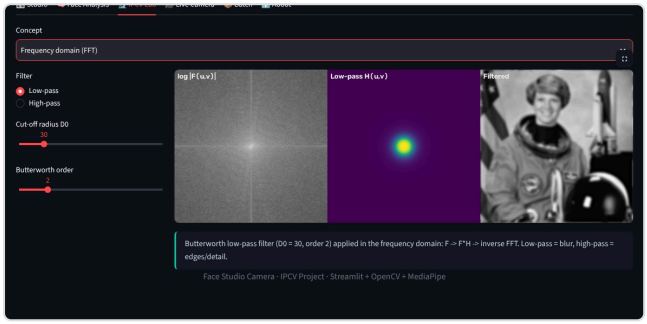


Figure 11.1 — Frequency domain: the spectrum, the Butterworth filter and the result.

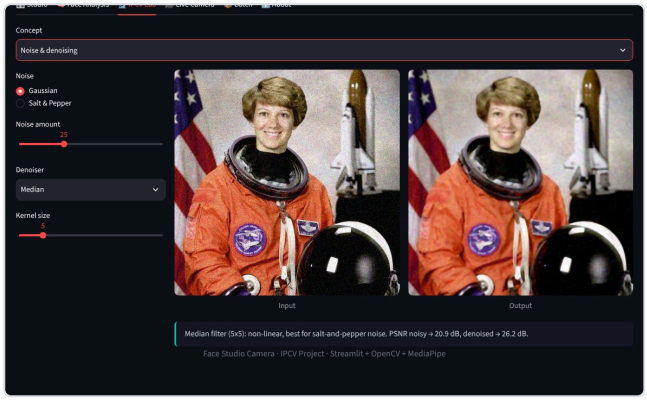


Figure 11.2 — Noise and denoising, with PSNR before and after.

Concept	How it is implemented	What to point out in the demo
Colour spaces	Split into RGB, HSV, LAB or YCrCb channels and show the three side by side (hue scaled ×255/180 for display).	Brightness sits in V and L*, separate from colour.
Histogram equalisation	Global `equalizeHist`, or CLAHE with adjustable clip limit and tile grid.	CLAHE brings out local detail without the harsh look; a higher clip limit gives more contrast and more noise.
Noise & denoising	Add Gaussian or salt-and-pepper noise (a fixed seed so it is repeatable), then denoise with mean, Gaussian, median, bilateral or non-local means. Prints the PSNR of the noisy and denoised images.	The median filter removes salt-and-pepper noise almost perfectly; mean and Gaussian only smear it. Median is non-linear: an extreme value never becomes the median.
Edge detection	Sobel (G_x, G_y, magnitude $\sqrt{(G_x^2 + G_y^2)}$), Prewitt (unweighted kernels), Laplacian (second derivative), Canny (gradient + non-maximum suppression + hysteresis thresholds).	Worked example: on a step from 10 to 200, Sobel's horizontal kernel [−1 0 1; −2 0 2; −1 0 1] gives (1 + 2 + 1) × (200 − 10) = 760 at the step and 0 on the flat parts. Canny gives thin, connected lines.
Thresholding	Otsu (automatic T that maximises the between-class variance), adaptive Gaussian (a different T for each 21×21 neighbourhood, offset 5), or a manual T.	Otsu prints the T it chose; adaptive copes with uneven lighting.

Morphology	On the Otsu mask with an elliptical structuring element: erosion, dilation, opening, closing, gradient, top-hat; size and iterations adjustable.	Opening removes small white specks; closing fills small black holes; gradient gives outlines.
Frequency domain	2-D FFT (`np.fft.fft2`), centred with `fftshift`; multiply by a Butterworth filter $H = 1 / (1 + (D/D_0)^{2n})$ (or $1 - H$ for high-pass); inverse FFT. Shows $\log\|F\|$, H and the result.	With $D_0 = 30$ and $n = 2$: H = 1.00 at the centre, 0.94 at D = 15, exactly 0.50 at D = 30 and 0.06 at D = 60. Low-pass = blur, high-pass = edges. Butterworth avoids the ringing of an ideal (sharp-cut) filter.
Geometric transforms	`getRotationMatrix2D` (rotation and scale about the centre) + `warpAffine` with bilinear interpolation and reflected borders; optional flips.	The 2×3 affine matrix M is printed, so you can show the rotation terms cos θ and sin θ.

12. Live video and batch mode

12.1 How live video reaches Python — the Live Camera tab

WebRTC is the browser technology behind video calls. `streamlit-webrtc` uses it to stream the webcam from the browser to the Python server, process each frame, and stream the result back:

1. The browser asks for camera permission and captures frames (we request 960×540, no audio).
2. Frames travel over a WebRTC connection to the Python process (on `localhost` when running on your laptop).
3. For every frame, streamlit-webrtc calls our `Proc.recv(frame)`: we convert it to a BGR NumPy array (`frame.to_ndarray(format="bgr24")`), process it, and wrap the result back into a video frame (`av.VideoFrame.from_ndarray`).
4. The processed stream plays in the page. With `async_processing=True`, processing runs in a separate thread, so a slow frame does not freeze the interface (frames are dropped instead).

The selected effect and the mirror checkbox are copied into the running processor on each rerun (`ctx.video_processor.lp.mode = live_mode`). Any error inside a frame is printed onto the frame instead of crashing the stream.

12.2 The per-frame engine — `live_effects.LiveProcessor`

One class serves both the browser tab and the desktop window. `process(frame)`:

1. Mirrors the frame horizontally if "selfie view" is on (`cv2.flip(frame, 1)`), so moving your right hand moves the right side of the picture.
2. Applies the chosen mode, one of 15: four AR overlays, head pose with blink counter, face mesh, face detection, privacy blur or pixelate, portrait blur, studio-colour background, beauty, cartoon, pencil sketch, original.
3. Measures its own time and draws the FPS on the frame. It keeps the last 30 frame times in a `deque`; FPS = 30 ÷ (sum of those 30 times). Example: 30 frames that took 1.0 s in total give 30 FPS. The window is cleared when you change mode, so the number always describes the current effect.

For speed, live mode uses single-pass detection (about 5 ms) and runs the mesh without the two-stage refinement, because at webcam distance faces are large.

12.3 The desktop live window — `live_camera.py`

The most reliable option for the demo, because it needs no browser or WebRTC. It opens the webcam with `cv2.VideoCapture(index)` at 1280×720, reads frames in a loop, processes them with the same `LiveProcessor`, and shows them with `cv2.imshow`. `cv2.waitKey(1)` reads the keyboard between frames: keys 1–9, 0 and a–e choose one of the 15 effects, n/p go to the next/previous effect, m toggles the mirror, s saves a snapshot to `saved_images/`, h hides the help list, and q or Esc quits. A clear message is printed if the camera cannot be opened (try `--cam 1`) or if the installed OpenCV has no GUI support.

12.4 Batch mode — the Batch tab

You upload several photos; for each one the app decodes it, runs the *current sidebar pipeline* with `run()` (the same function the Studio uses), encodes the result as PNG, and writes it into a ZIP archive built in memory (`zipfile.ZipFile` over a `BytesIO` buffer, so nothing touches the disk). A progress bar advances per photo, and a table lists each file's size, number of stages, time and SSIM against its original. If the pipeline is empty, the tab says so and suggests a preset.

13. The pipeline engine, user interface and engineering

13.1 Why the stages run in this order

Order	Reason
1 Enhance first	Global tone fixes should affect everything that follows, including the skin and the background.
2 Beauty before 3 Background	Skin smoothing uses its own face mask; if the background were blurred first, the bilateral filter would sample blurred pixels at the face border.
3 Background before 4 Style	The style then applies to the whole new composite, so a replaced backdrop and the person get the same cartoon or sepia look.
5 Finishing after the style	A vignette and grain are "lens and film" effects, applied on top of the final look, like a real camera.
6 AR after the style	The sunglasses stay crisp instead of being cartoonised or sketched.
7 Privacy last	Nothing can draw over a blurred face afterwards, so anonymisation can never be undone by a later stage.

13.2 Settings, presets and the engine — `pipeline.py`

- **Settings** are one dictionary (`DEFAULTS`) with 22 keys, such as `"brightness": 0` and `"bg_mode": "None"`. `settings_from(overrides)` copies the defaults and applies changes.
- **Presets** are just small override dictionaries. For example, Portrait Pro = auto-enhance + skin smoothing 0.5 + brighten 6 + portrait blur 18 + vignette 0.6 + sharpness 0.4. The 7 presets are Portrait Pro, Vintage, Noir, Cartoon Pop, Backdrop, Party Mode and Privacy Safe.
- **Lazy getters.** `run_pipeline` receives *functions* (`get_landmarks`, `get_mask`, `get_faces`) rather than results, and calls them only if an active stage needs them. With only brightness enabled, no neural network runs at all.
- **Timing and notes.** Each stage is timed with `time.perf_counter()` and recorded in a `PipelineResult`, which feeds the per-stage timing table. Helpful notes such as "No face found – face effects were skipped" are collected and shown as warnings instead of errors.
- **One engine everywhere.** The Studio, Batch, live video and the benchmark all call the same code, so a fix in one place fixes them all. The `Models` class gives scripts outside Streamlit the same lazily-loaded models.

13.3 The user interface — `app.py`

Sliders that presets can move

Every pipeline widget has a key in `st.session_state` with the prefix `s_` (for example `s_brightness`). Streamlit does not allow code to change a widget's value after the widget has been drawn in the same run, so the preset buttons use **callbacks**: `st.button(name, on_click=apply_preset, args=(name,))`. A callback runs *before* the next rerun draws the widgets; it writes the preset's values into the session state, and the sliders then appear at the new positions. Reset works the same way with the defaults. On the very first run an `initialised` flag fills in the defaults once.

The before/after slider — `compare_slider()`

Streamlit has no built-in compare slider, so we built one in about 40 lines of HTML, CSS and JavaScript, embedded with `components.html` (no extra package). Both images are encoded as base64 JPEG and stacked in the same box: the "after" image fills the box, and the "before" image sits on top inside a `div` whose width is a percentage of the box. Dragging the handle sets that percentage (mouse or touch), revealing more or less of the "before" image. The box keeps the photo's aspect ratio, so the handle lines up with the picture.

Other details

- **Version-tolerant helpers.** Newer Streamlit versions renamed `use_container_width=True` to `width="stretch"`. `show()` and `table()` try the new form and fall back to the old one, so the app works on both.
- **Friendly model errors.** If a model file is missing and cannot be downloaded, a `ModelMissingError` is caught and the tab shows "run `python download_models.py`" instead of a stack trace.
- **Detection settings.** The sidebar's multi-scale checkbox and tile-threshold slider are part of the cache key, so changing them re-runs detection with the new values.
- **Configuration.** `.streamlit/config.toml` sets the dark theme and turns off Streamlit's usage statistics, so the app does not try to send them during an offline demo.

13.4 Models, installation and testing

- **Model downloads** (`download_models.py`): the three official files are fetched from Google's MediaPipe model storage into `models/`. Each download goes to a temporary `.part` file and is renamed only when complete (`os.replace`, which is atomic), so an interrupted download never leaves a broken model that would crash the app later. The environment variable `FSC_MODELS_DIR` can point to another folder.
- **Why `requirements.txt` has no OpenCV line:** MediaPipe already installs `opencv-contrib-python` (which includes the GUI used by `live_camera.py`). Listing a second OpenCV package would install two conflicting copies of `cv2`.
- **Self-check** (`check_setup.py`): prints PASS/FAIL for each package, each model file, each model actually loading and producing output on the sample, every preset, head pose, the webcam and the OpenCV GUI. It exits with an error code if anything required fails.
- **Unit tests** (`python -m pytest -q`): 8 tests. They run on random synthetic images, so most need no models; the model test is skipped automatically when the model files are absent.

14. Limitations and honest notes

- **Small faces:** even with tiling, 4 of the 7 small, dark faces in the Addams Family test image were missed. A detector trained for long-range faces would help further.
- **False boxes:** tiling produced 13 extra boxes in 84 degraded trials, mostly low-confidence hits on noisy textures. Raising the tile threshold trades them for missed faces.
- **Segmentation edges:** fine hair is softened or cut, because the mask is computed at low resolution and feathered.
- **Head-pose bias:** pitch reads about −12° on a near-frontal face, so the angles are best used for changes (nodding, turning) rather than as absolute truth.
- **Small test set:** 8 images give indicative results only; a large labelled dataset such as WIDER FACE would give tighter numbers.
- **How the code was tested:** during development, Google's model-download server was unreachable from the build machine, so testing used earlier MediaPipe releases of the same detector and segmentation networks, with the legacy face-mesh model run directly. All features, the 8 unit tests and the web interface passed. Running `check_setup.py` and `benchmark.py` on the demo laptop with the official models confirms the final numbers.
- **Online deployment:** everything except `live_camera.py` can run on a host such as Streamlit Community Cloud, but the Live Camera tab would need STUN/TURN servers configured, and the server needs the `libgl1` system package for OpenCV.

Appendix A — Formula sheet

Operation	Formula
Greyscale	$Y = 0.299R + 0.587G + 0.114B$
Contrast	$F = 259(c + 255) / (255(259 - c))$; $out = F(in - 128) + 128$
Gamma	$s = 255\,(r / 255)^{1/\gamma}$
Unsharp mask	$out = (1 + a)\,I - a\,G_\sigma(I)$
Histogram equalisation	$s = 255 \cdot CDF(r)$
Bilateral weight	$w = \exp(-d^2 / 2\sigma_s^2) \cdot \exp(-\Delta^2 / 2\sigma_r^2)$
Alpha compositing	$O = \alpha\,I + (1 - \alpha)\,B$
Face frame	$pixel = origin + d\,(u\,e_1 + v\,e_2)$; $e_2 = (-e_{1y}, e_{1x})$
IoU	$\lvert A \cap B \rvert / \lvert A \cup B \rvert$
Head pose	$yaw = \operatorname{atan2}(-z_x, z_z)$; $pitch = \operatorname{atan2}(z_y, \sqrt{z_x^2 + z_z^2})$; $roll = \operatorname{atan2}(x_y, x_x)$
Eye aspect ratio	$(\lVert p2 - p6 \rVert + \lVert p3 - p5 \rVert) / (2\lVert p1 - p4 \rVert)$; closed below 0.20
Mouth aspect ratio	$\lVert p13 - p14 \rVert / \lVert p61 - p291 \rVert$; open above 0.35
PSNR	$10 \log_{10}(255^2 / MSE)$

SSIM	$(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2) / ((\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2))$
Entropy	$-\Sigma\, p_i \log_2 p_i$
Sobel magnitude	$\sqrt{(G_x^2 + G_y^2)}$
Butterworth low-pass	$H = 1 / (1 + (D / D_0)^{2n})$; high-pass = $1 - H$

Appendix B — Glossary

Term	Meaning
Anchor	A fixed candidate box position a detector checks for an object.
Bilateral filter	A smoothing filter that weights neighbours by distance and by similarity, so edges are preserved.
CLAHE	Contrast-Limited Adaptive Histogram Equalisation: equalisation per tile, with a cap on contrast.
CNN	Convolutional neural network: layers of learned filters, the basis of modern vision models.
Confidence mask	A per-pixel probability (0–1) that the pixel belongs to a class, here "person".
Dilation / erosion	Morphological operations that grow / shrink white regions of a mask.
EXIF	Metadata in photo files, including the camera's rotation.
Feathering	Blurring a mask's edge so a composite blends smoothly.
Gram–Schmidt	Making vectors perpendicular by subtracting their components along earlier vectors.
Integral image	A table of cumulative sums allowing any rectangle sum in 4 lookups (Haar).
IoU	Intersection over Union: the overlap of two boxes, from 0 to 1.
Landmark	A specific, consistently numbered point on the face (for example 1 = nose tip).
LUT	Lookup table: precomputed outputs for all 256 input values.
NMS	Non-maximum suppression: keeping only the best of overlapping detections.
Point operation	A pixel operation whose output depends only on that pixel's own value.
Segmentation	Labelling every pixel with a class (person or background).
TFLite	TensorFlow Lite: a compact format for running neural networks on CPUs and phones.
Unsharp masking	Sharpening by adding back the difference between an image and its blur.
WebRTC	The browser technology for real-time audio/video streams.